

Verification Facade: Masquerading Insecure Cryptographic Implementations as Verified Code

Nadim Kobeissi

Symbolic Software, Paris, France
`nadim@symbolic.software`

Abstract. Hax is a verification pipeline that translates a subset of Rust into F^* , enabling machine-checked proofs of panic freedom and functional correctness for cryptographic implementations being developed in partnership with Google and tested in Signal’s post-quantum protocol. We study whether hax’s translation preserves the security properties it claims to verify. Through a structural analysis of its 35-phase transformation engine, F^* proof libraries, and specification API, we identify three classes of semantic gap between the Rust source and the F^* verification target: *translation infidelity*, where pipeline transformations distort security-relevant semantics; *unverifiable trust boundaries*, where operations are axiomatized without postconditions; and *specification gaming*, where escape hatches inject unproven facts into the verification context. We demonstrate each class through five proof-of-concept exploits against ML-DSA (FIPS 204), ML-KEM (FIPS 203), Ed25519 (FIPS 186-5), and ChaCha20 (RFC 8439). Every exploit meets a strict criterion: the Rust code compiles, passes functional tests, and extracts to F^* without warnings, while harboring a security gap invisible to testing. We distinguish three gradations: *facade* gaps where the F^* model actively diverges from Rust semantics, a *conditional* gap dependent on the compilation mode, and a *scope* gap where the model is faithful but cannot cover a critical property. We call the resulting phenomenon a *verification facade*: verification that is performed but covers less than it appears to cover.

Keywords: high-assurance cryptography · formal verification · verification boundaries · cryptographic engineering

1 Introduction

The formal verification of cryptographic implementations has matured from just a simple numbers game to a deployment requirement. ML-KEM [11] and ML-DSA [12] implementations verified via hax [2] are being developed in partnership with Google [9] and tested in Signal’s PQXDH protocol [17]. The pipeline is straightforward: write cryptographic code in a subset of Rust, annotate it with specifications, extract it to F^* [18], and discharge proof obligations with an SMT

Table 1. Proof-of-concept exploits. Each produces compilable, test-passing Rust that extracts to F^* with a security-relevant gap. **Str.:** F = facade (model diverges); C = conditional; S = scope (model faithful but incomplete).

	Primitive	Gap	Class	Str.	Standard
E1	ML-DSA	Rejection sampling	I-A	F	FIPS 204
E2	ML-KEM	Arithmetic model	I-C	C	FIPS 203
E3	Ed25519	Scalar clamping	III-A	F	FIPS 186-5
E4	ML-KEM	Coefficient sampling	I-A	F	FIPS 203
E5	ChaCha20	Rotation semantics	II-B	S	RFC 8439

solver. If F^* typechecks the extracted code, the implementation is verified with respect to the specifications provided.

A companion paper [5] studied what happens when the *verification boundary*—the interface between verified and unverified code—is not communicated clearly, finding thirteen vulnerabilities in Cryspen’s libcrux that escaped formal verification. Those findings concerned code *outside* the verified perimeter.

We study the perimeter itself. Can the hax pipeline produce verification output that does not mean what it appears to mean—not because proofs are skipped, but because the model that F^* verifies diverges from the program that Rust executes? We find that it can. Through a structural analysis of hax’s 35-phase transformation engine, its F^* proof libraries, and its specification API, we identify three classes of gap (Figure 1):

1. **Translation infidelity:** the pipeline distorts security-relevant semantics; the F^* model does not match the Rust code.
2. **Unverifiable trust boundaries:** operations are axiomatized without definition; F^* cannot reason about their behavior.
3. **Specification gaming:** escape hatches inject unproven facts into the verification context.

We demonstrate these gaps through five proof-of-concept exploits (Table 1). Each exploit meets a strict inclusion criterion: the Rust code must compile, pass functional tests, and extract to F^* without warnings, while containing a security gap invisible to testing. We distinguish three gradations of severity: *facade gaps*, where the F^* model actively diverges from Rust (E1, E3, E4); a *conditional gap*, where the divergence depends on the compilation mode (E2); and a *scope gap*, where the model is faithful but structurally cannot cover a critical property (E5).

We define a *verification facade* as a system where code passes through a formal verification pipeline, producing structurally valid output, yet the verified model either diverges from the deployed code’s behavior, depends on unenforceable assumptions, or cannot express a security-critical property. Unlike *verification theatre* [5]—where verification is claimed but not performed—a facade involves verification that is performed but covers less than it appears to cover.

Contributions. We present the following contributions:

What Rust executes		What F [*] sees
<i>Class I Translation infidelity</i>		
<code>while</code> loop iterates until condition met	≠	Loop body is proof-inert; fuel = 0 (E1, E4)
Release mode: + wraps mod 2 ⁿ	≠	+! panics on overflow (E2)
<i>Class II Unverifiable trust boundaries</i>		
<code>rotate_left(16)</code> computes correct rotation	≈	<code>assume val</code> : no definition to verify against (E5)
<i>Class III Specification gaming</i>		
<code>assume!(P)</code> compiles to <code>()</code>	≠	<i>P</i> accepted as axiom without proof (E3)

Fig. 1. The verification facade. Each row contrasts Rust runtime behavior (left) with what the F^{*} model sees (right). ≠ denotes active divergence; ≈ denotes faithful but incomplete coverage. E1–E5 reference the exploits in Section 3.

1. A structural analysis of hax’s Rust-to-F^{*} pipeline identifying three classes of semantic gap—translation infidelity, unverifiable trust boundaries, and specification gaming—with source-level evidence drawn from the 35-phase transformation engine, the F^{*} proof libraries, and the specification API.
2. Five proof-of-concept exploits against ML-DSA (FIPS 204), ML-KEM (FIPS 203), Ed25519 (FIPS 186-5), and ChaCha20 (RFC 8439), each meeting our strict inclusion criteria: the Rust code compiles, passes functional tests, and extracts to F^{*} without warnings, while harboring a security-relevant gap invisible to testing.
3. A fine-grained classification distinguishing fixable bugs, engineering tradeoffs, fundamental limitations, and intentional features, with concrete mitigations for each category and a discussion of which gaps can be addressed within hax’s current architecture and which require a different verification approach.

2 The Hax Pipeline

Hax [2] translates a subset of Rust into F^{*} [18] through three stages.

Frontend. A custom compiler driver (`hax-driver`) hooks into `rustc` and extracts the Typed High-Level Intermediate Representation (THIR) for each item. THIR preserves high-level structure—loops, matches, closures—that would be lost in MIR’s control-flow graph. This design choice is consequential: hax operates on a representation where references are still present, borrow-checker constraints are still implicit, and `Drop` glue has not been elaborated. The extracted AST

is serialized as JSON and passed to the engine via stdin; the communication protocol is defined in `hax_types::engine_api`.

Transformation engine. An OCaml engine applies 35 sequential phases, each eliminating or transforming a language feature. The engine’s AST is parameterized by a *feature set*: 30 type-level booleans that control which constructs may appear. Each phase is a functor that turns features off, progressively simplifying toward a purely functional representation. Figure 2 summarizes the six functional groups.

To illustrate concretely: a Rust function `fn f(x: &mut u32) { *x += 1; }` undergoes the following transformations. `phase_direct_and_mut` (Group 4) converts the `&mut` parameter into a return value: the function’s return type changes from `()` to `u32`, and the caller receives the modified value. `phase_drop_references` (Group 3) then erases the `&` from the parameter type: `&mut u32` becomes `u32`. `phase_local_mutation` (Group 4) threads any remaining mutable locals through the expression as immutable tuple accumulators. The result is a purely functional F^* definition that takes a `u32`, returns `u32`, and contains no mutation.

For loops, the transformation is more involved. A `for i in 0..n { body }` becomes a call to `fold_range 0 n inv init (fun acc i → body')`, where `inv` is the loop invariant (from `loop_invariant!`, or `fun _ → true` if absent) and `init` is the initial accumulator (a tuple of all mutable variables used in the loop). A `while cond { body }` becomes a call to `while_loop inv cond fuel init (fun acc → body')`, where `fuel` is the termination measure (from `loop_decreases!`, or constant 0 if absent—the bug exploited in E1 and E4).

By the time the AST reaches the backend, the following have been eliminated: mutable variables, references, raw pointers, lifetimes, `Box<T>`, block expressions, match guards, early exits, all loop forms, the `?` operator, and `as` patterns. What remains is a purely functional language with algebraic data types, pattern matching, monadic bindings, fold-based iteration, and typeclass-resolved trait dispatch.

The feature-witness system ensures that the pipeline *cannot accidentally emit* a node for a disabled feature—a real engineering strength. However, the system guarantees only *structural* correctness (the output has the right shape), not *semantic* preservation (the output has the same meaning).

The full pipeline is defined as an OCaml functor composition in `fstar_backend.ml`:

F^ backend.* The backend (`fstar_backend.ml`, 2,091 lines) translates the simplified AST to F^* surface syntax. A brief primer on F^* is helpful here. F^* is a dependently typed programming language and proof assistant: every value can carry a *refinement type* that constrains its range, and the SMT solver Z3 must discharge these constraints at typecheck time. For example, `hax` models `u8` as `int_t U8`—an integer x with the refinement $0 \leq x \leq 255$. The F^* type `Prims.Pure t (requires P) (ensures Q)` means: “this function returns a value of type `t`, requires precondition `P` to hold on entry, and guarantees postcondition `Q` on the return value.” Z3 must *prove* both that `P` holds at every call site and that `Q`

#	Group	Key phases (security-critical in bold)
1–6	Reject & normalize	Reject.RawOrMutPointer, Specialize, Drop_sized_trait
7–12	Reconstruct	Reconstruct_asserts, Reconstruct_for/while_loops
13–21	Erase Rust semantics	Drop_references , Explicit_conversions, Hoist
22–25	Functionalize	Local_mutation , Functionalize_loops
26–30	Spec. & types	Traits_specs, Newtype_as_refinement
31–35	Finalize	Bundle_cycles, SubtypeToInputLanguage

Fig. 2. The 35-phase transformation pipeline, organized into six groups. Bold phases perform the deepest semantic transformations and are implicated in the gaps we identify.

follows from the function body. If Z3 cannot discharge a proof obligation, the code does not typecheck—the verification fails.

Hax translates Rust’s standard `+` operator to F^* ’s `+`!, which carries a refinement requiring the result to fit the integer type (e.g., for `u8`: $a + b \leq 255$). This means that every addition generates a proof obligation that the sum does not overflow. The wrapping variant `wrapping_add` becomes `+`., which uses modular arithmetic and requires no overflow proof. Traits become F^* records; implementations become typeclass instances.

The keyword `assume val` declares a function with a type signature but *no implementation*. F^* accepts the function’s existence on faith: it can be called, and callers may rely on its type, but there is no body for Z3 to verify. This is F^* ’s mechanism for axiomatization—the analogue of an unproven axiom in mathematics. If the axiom is wrong (the actual implementation does not satisfy the declared type), every proof that depends on it is unsound.

Trusted computing base. For hax’s verification claims to hold, three components must be correct:

- The 35 transformation phases (unverified OCaml; no mechanized semantic-preservation proof).
- The F^* proof libraries, which axiomatize several operations via `assume val`: bit rotations, byte-order conversions, `while_loop_return`, range-based array updates, and several fold variants.
- User annotations: `#[opaque]` (hides a function body), `assume!` (injects an axiom), `fstar::replace` (substitutes a definition), and `verification_status(lax)` (disables proofs).

```

1 module TransformToInputLanguage =
2   Phases.Reject.RawOrMutPointer(Features.Rust)
3   |> Phases.Specialize
4   |> Phases.Simplify_question_marks
5   |> Phases.Reconstruct_asserts
6   |> Phases.Reconstruct_for_loops
7   |> Phases.Reconstruct_while_loops
8   |> Phases.Drop_references      (* erases &T -> T *)
9   |> Phases.Explicit_conversions
10  |> Side_effect_utils.Hoist
11  |> Phases.Local_mutation      (* mut -> state-passing *)
12  |> Phases.Rewrite_control_flow
13  |> Phases.Functionalize_loops  (* while -> fold; E1, E4 *)
14  |> Phases.Traits_specs
15  |> Phases.Newtype_as_refinement
16  |> Phases.Bundle_cycles
17  |> SubtypeToInputLanguage

```

Listing 1.1. The 35-phase pipeline (abbreviated). Comments mark phases implicated in our exploits.

Hax claims to verify panic freedom, functional correctness, and termination. It explicitly does *not* claim translator soundness, memory safety (delegated to Rust’s borrow checker), constant-time behavior, or verification of compiled binaries.

What “extraction succeeds” means. Running `cargo hax` into `fstar` on a Rust crate produces F^* source files in a `proofs/fstar/extraction/` directory. If the command completes without errors, every Rust function in the crate has been translated to an F^* definition. We additionally ran the F^* typechecker (v2025.03.25) in lax mode on all five PoCs; all pass, confirming the extracted code is structurally valid F^* that the typechecker accepts. Throughout this paper, “extraction succeeds” means both that hax produces F^* without error and that the output lax-typechecks.

Quantitative TCB. We measured the axiomatization footprint in the proof libraries. `Core_models.Num.fst` contains 108 `assume val` declarations (9 operations $\times$ 12 integer types: `rotate_left/right`, `leading_zeros`, `ilog2`, `from/to_be/le_bytes`, `from_str_radix`) alongside 450 defined operations. Roughly 19% of the integer model is axiomatized. `Rust_primitives.Hax.fst` adds 5 more `assume val` declarations (4 range-based array updates and `while_loop_return`). `Rust_primitives.Integers.fsti` defines 144 integer operations with full refinement types; `rotate_left` and `rotate_right` are declared (`val`) without implementation.

2.1 What Hax Verifies Correctly

The gaps in this paper should not obscure what hax does well. For code that fits its extraction model—deterministic `for` loops over bounded ranges, pure

```

1  #[requires(i64::from(value) >= -BARRETT_R
2      && i64::from(value) <= BARRETT_R)]
3  #[ensures(|result|
4      result > -FIELD_MODULUS
5      && result < FIELD_MODULUS
6      && result % FIELD_MODULUS == value % FIELD_MODULUS)]
7  pub fn barrett_reduce(value: i32) -> i32 {
8      let t = i64::from(value) * BARRETT_MULTIPLIER;
9      let t = t + (BARRETT_R >> 1);
10     let quotient = (t >> BARRETT_SHIFT) as i32;
11     let sub = quotient * FIELD_MODULUS;
12     value - sub
13 }

```

Listing 1.2. Barrett reduction from hax's own examples (`examples/barrett/src/lib.rs`). The `requires/ensures` clauses become real F^* proof obligations—not axioms.

arithmetic with explicit bounds, array transformations with statically known indices—hax provides genuine verification.

Integer overflow. Machine integers are modeled as refinement types over mathematical integers with exact range constraints: `u8` is $\{x : \mathbb{Z} \mid 0 \leq x \leq 255\}$. Every use of `+`, `*`, or `/` generates a proof obligation that the result stays in range. Array indexing via `. [ i ]` requires a bounds proof ($0 \leq i < \text{len}$). These are not vacuous checks: they catch real overflow and out-of-bounds errors that testing routinely misses.

Bounded loops. `for` loops over bounded ranges (`for i in 0..n`) extract to `fold_range`, which is *defined* (not axiomatized) with a recursive implementation in `Rust_primitives.Hax.Folds.fsti`. Loop invariants provided via `loop_invariant!` become genuine proof obligations. The Barrett reduction and Kyber compression examples in hax's own repository demonstrate that NTT butterflies, modular reductions, and polynomial operations can be meaningfully verified:

Postconditions. Unlike `assume!`, the `ensures` clause in Listing 1.2 is a *proof obligation*: F^* must verify that the function body satisfies it. If the body violated the postcondition (e.g., returning a value $\geq q$), F^* would reject the code. We confirmed this experimentally: a function annotated `#[ensures(|r| r > 0)]` that returns 0 for some input produces F code that the typechecker rejects. This is a genuine strength—`ensures` clauses are verified, not trusted.

Mutable references. Hax's `phase_direct_and_mut` correctly transforms `&mut` parameters into return values. For example, an in-place buffer XOR function taking `&mut [u8]` extracts to an F^* function returning `(t_Slice u8)`: the modified slice is threaded as a return value, and callers see the mutation. We verified

this experimentally: the extracted F^* correctly models the mutation as a functional update.

3 The Verification Facade

We organize the gaps into three classes, determined by where the divergence originates. For each class, we first define the gap, then demonstrate it with a concrete exploit. Every exploit meets the inclusion criteria stated in Section 1 and visualized in Figure 3: correct Rust, successful extraction, security-relevant divergence, invisible to testing.



Fig. 3. Exploit inclusion criteria. Every proof-of-concept must pass all four gates: it compiles as valid Rust, extracts to F^* without warnings, introduces a security-relevant divergence between model and code, and escapes detection by functional tests.

We distinguish two kinds of gap. A *facade gap* (E1, E3, E4) arises when the F^* model actively diverges from Rust semantics: the verified model is wrong. A *scope gap* (E5) arises when the model is faithful but the verification cannot cover a critical property because the relevant operation is axiomatized. E2 is a *conditional facade gap*: the model matches debug-mode Rust but diverges in release mode.

3.1 Class I: Translation Infidelity

The pipeline’s 35 phases transform Rust into a purely functional representation. Some transformations lose or distort security-relevant semantics. The F^* proofs are correct *for the model they verify*, but the model does not match the code that runs.

I-A: While-loop ghosting. Hax converts **while** loops into calls to a **while_loop** combinator defined in the F^* proof library. The combinator takes five arguments: an invariant, a condition, a *fuel function*, an initial accumulator, and the loop body. The fuel function maps each accumulator state to a natural number; the combinator requires that fuel strictly decreases on each iteration (**fuel(next)** < **fuel(current)**), providing a termination proof. The loop body’s type encodes this requirement as a refinement:

When a **while** loop lacks a **loop_decreases!** annotation, **phase_functionalize_loops.ml** (lines 276–293) must still generate a fuel function. It defaults to constant 0: **fun _ → from_machine(mk_u32 0)**. With this fuel, the body’s


```

1 let while_loop #acc_t
2   (inv: acc_t → Type0)
3   (condition: (c:acc_t {inv c}) → bool)
4   (fuel: (a:acc_t{inv a} → nat))
5   (init: acc_t {inv init})
6   (f: (i:acc_t{inv i /\ condition i}
7       → o:acc_t{inv o /\ fuel o < fuel i}))
8   : (res: acc_t {inv res /\ not (condition res)})

```

Listing 1.3. The `while_loop` combinator from `Rust_primitives.Hax.fst`. The body (parameter `f`) must return a state whose fuel is strictly less than the input’s.

refinement becomes `fuel o < fuel i = 0 < 0 = False`. The refinement is unsatisfiable: any call to the body must produce a value satisfying `False`, which is impossible. We call the result *proof-inert*: the loop body exists syntactically but cannot support any verification claim about the accumulator.

The critical subtlety is that this is *not* a hard error. The F^* code parses, and `cargo hax into fstar` succeeds without warnings. A developer who does not attempt a proof about the loop—or who attributes Z3’s failure to specification complexity—will never discover that the loop body is proof-inert.

I-B: Reference erasure. Phase 16 erases all reference types: $\&T \rightarrow T$, with lifetime parameters removed. Interior mutability (`Cell<T>`, `RefCell<T>`) appears immutable in F^* . Rust’s ownership discipline has no F^* representation.

I-C: Deployment model divergence. Hax translates Rust’s `+` to F^* ’s `+`! (strict, with overflow proof obligation), matching debug-mode semantics. In release mode, overflow wraps silently. Code verified panic-free in F^* will not panic in debug mode but may produce wrong results in release mode.

E1: ML-DSA Rejection Sampling (Facade, I-A) ML-DSA signing [12] uses rejection sampling (Algorithm 7, steps 10–32). The signer computes $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and checks that $\|\mathbf{z}\|_\infty$ stays below $\gamma_1 - \beta$. Failing candidates leak information about $\mathbf{s}_1$ and must be discarded. This loop is what makes ML-DSA signatures zero-knowledge.

The Rust implementation¹ (crate `e1_mldsa_rejection`) is straightforward (Listing 1.4): a `while` loop generates candidate signatures and retries on rejection. The code compiles, the rejection loop runs correctly, and sign-then-verify tests pass. But the loop lacks a `loop_decreases!` annotation, so hax generates constant fuel 0.

Hax translates this loop into a call to the `while_loop` combinator shown in Listing 1.3. The extracted F^* is shown in Listing 1.5. To understand the gap, follow the five arguments to `while_loop`:

¹ All PoC source code and pre-extracted F^* output are available at <https://github.com/symbolicsoft/verification-facade>.

```

1 pub fn sign_message(s1: &SecretS1, seed: u64) -> SignatureZ {
2     let mut nonce: u32 = 0;
3     let mut z = generate_mask(seed, nonce);
4     z = compute_z(&z, s1);
5     while !check_norm_bound(&z) { // rejection sampling
6         nonce = nonce.wrapping_add(1);
7         z = generate_mask(seed, nonce);
8         z = compute_z(&z, s1);
9     }
10    z // Rust: always passes check_norm_bound
11 }

```

Listing 1.4. E1: Rust source (simplified). The **while** loop on line 5 is correct rejection sampling. No `loop_decreases!` annotation is present.

```

1 let sign_message (s1: t_SecretS1) (seed: u64) =
2   let z = compute_z (generate_mask seed 0) s1 in
3   let (nonce, z) =
4     Rust_primitives.Hax.while_loop
5     (fun _ → true) (* 1. invariant: trivially true *)
6     (fun (_, z) → ~(check_norm_bound z)) (* 2. condition *)
7     (fun _ → Int.from_machine (mk_u32 0)) (* 3. fuel = 0 *)
8     (0, z) (* 4. initial accumulator *)
9     (fun (nonce, z) →
10      let nonce = wrapping_add nonce 1 in
11      let z = compute_z (generate_mask seed nonce) s1
12      in (nonce, z))
13   in z

```

Listing 1.5. E1: Extracted F^* for ML-DSA signing.

1. The **invariant** (line 5) is `fun _ → true`—trivially satisfied, imposing no constraint.
2. The **condition** (line 6) checks whether the current signature candidate fails the norm bound, mirroring the Rust **while** condition.
3. The **fuel function** (line 7) returns `from_machine(mk_u32 0)`—the integer 0, regardless of the accumulator state. This is the bug: the fuel should decrease on each iteration (e.g., tracking the remaining nonce space), but `hax` generates a constant because no `loop_decreases!` annotation was provided.
4. The **initial accumulator** (line 8) is the first candidate `(0, z)`.
5. The **loop body** (lines 9–12) generates a new mask and computes the next candidate—exactly mirroring the Rust loop body.

The problem is in argument 3. Recall from Listing 1.3 that the body’s type requires `fuel o < fuel i`: the output’s fuel must be strictly less than the input’s. With constant fuel 0, this becomes `0 < 0 = False`. In F^* ’s type system, a function whose return type includes `False` can never be called—it is *dead code for verification purposes*. The loop body exists syntactically but Z3 cannot use it to prove anything about how the accumulator changes. In effect, any proof that

tries to conclude “the returned $\mathbf{z}$ satisfies `check_norm_bound`” would require discharging $0 < 0$, which Z3 correctly rejects.

Cryptographic consequence. The Rust code is correct: the rejection loop works, and the function only returns candidates that pass the norm bound. But the F^* model cannot verify this property. ML-DSA’s security relies on the Fiat-Shamir with Aborts paradigm [8]: rejection sampling ensures that $\mathbf{z}$ ’s distribution is statistically independent of $\mathbf{s}_1$. This is the property that the security proof requires and that a developer would want to establish formally. Because the loop is proof-inert, F^* cannot express or verify it—there is no way to prove in F^* that the function only returns candidates that pass the norm bound. The Rust implementation happens to be correct, but the verification provides no assurance of this. (For context: without proper rejection, $\mathbf{z}$ ’s distribution would be centered near $c\mathbf{s}_1$, and over approximately 1000 signatures, lattice techniques could recover $\mathbf{s}_1$ from the statistical bias.)

A stronger variant. Because the `while` loop is proof-inert, F^* cannot distinguish a correct rejection loop from a *broken* one. An adversarial developer could invert the loop condition—`while check_norm_bound(&z)` instead of `while !check_norm_bound(&z)`—causing the function to return the first candidate that *fails* the norm bound, directly leaking $\mathbf{s}_1$. Alternatively, the loop body could reuse the same nonce on every iteration instead of incrementing it, eliminating re-randomization. In both cases, the extracted F^* is structurally identical: the same `while_loop` call with the same `fuel=0`, and the same inability to prove any property about the output. The inversion would cause a conforming verifier to reject some signatures (since FIPS 204, Algorithm 8, step 13 checks the same bound), so functional testing would eventually detect the bug—but the point is that F^* provides no earlier warning. The verification facade is the same whether the underlying code is correct or broken: the `while` loop is proof-inert either way.

Root cause. A silent default in `phase_functionalize_loops.ml` (lines 276–293): when `loop_decreases!` is absent, the code generates constant fuel 0 via `from_machine(mk_u32 0)`. The extracted F^* looks structurally identical to a correctly annotated loop—the five-argument structure is the same and the fuel function has the right type signature—so only by examining the fuel function’s *value* or by attempting a proof about the accumulator would a developer discover the gap.

E2: ML-KEM Barrett Overflow (Conditional, I-C) ML-KEM [11] performs all polynomial arithmetic modulo $q = 3329$. Rather than computing the expensive modulo operation directly, implementations use *Barrett reduction*: approximate the quotient $\lfloor \text{value}/q \rfloor$ via integer multiplication and a shift, then subtract $\text{quotient} \times q$. The approximation is exact when the input is bounded: $|\text{value}| \leq 2^{26}$ guarantees the quotient is correct.

```

1 let barrett_reduce (value: i32)
2   : Prims.Pure i32
3   (requires
4     (cast value <: i64) >=. (neg v_BARRETT_R)
5     && (cast value <: i64) <=. v_BARRETT_R)
6   (ensures fun result →
7     result >. (neg v_FIELD_MODULUS)
8     && result <. v_FIELD_MODULUS) =
9   let t = (cast value <: i64) *! v_BARRETT_MULTIPLIER in
10  let t = t +! (v_BARRETT_R >>! mk_i32 1) in
11  let quotient = cast (t >>! v_BARRETT_SHIFT) <: i32 in
12  value -! (quotient *! v_FIELD_MODULUS)

```

Listing 1.6. E2: Barrett reduction (excerpt). The **requires** clause bounds the input; the **ensures** clause asserts the output is in range. All arithmetic uses strict operators (***!**, **+!**, **-!**).

The PoC (crate `e2_mlkem_barrett`) implements this reduction with correct preconditions. The code compiles, passes tests in debug mode, and extracts to F^* :

Reading the F^* listing line by line:

- Lines 3–5: the **requires** clause bounds the input to $|value| \leq 2^{26}$, ensuring intermediates fit.
- Lines 6–8: the **ensures** clause promises the output is in $(-q, q)$ and congruent to the input modulo q .
- Line 10: the multiplication uses ***!**— F^* ’s strict operator, which carries a refinement requiring the product to fit in `i64`. Under the precondition, this holds: $2^{26} \times 20159 < 2^{63}$.
- Line 13: the final subtraction also uses **-!** (strict).

The key observation: every ***!** and **-!** in the F^* output corresponds to a plain ***** and **-** in the Rust source. In Rust’s debug mode, these operators panic on overflow—matching F^* ’s strict semantics. But in *release mode*, which is how production code is compiled, Rust performs two’s-complement wrapping instead: the overflow is silent, the result is mathematically wrong, and no panic occurs.

This gap is *conditional*: it manifests only when code is compiled in release mode and a caller violates the precondition—conditions common in deployment but absent from debug-mode testing. The Barrett quotient becomes wrong, and the result is not congruent to $value \bmod 3329$. The F^* proof establishes: “if the precondition holds, the arithmetic is correct; if it doesn’t, the program panics.” Release mode introduces a third possibility—precondition violated, no panic, silently wrong result—that the F^* model does not represent.

Cryptographic consequence. Incorrect Barrett reductions corrupt NTT polynomial arithmetic, causing decryption failures that break ML-KEM’s IND-CCA security via the Fujisaki–Okamoto transform.

Root cause. The backend always generates strict arithmetic (**+!**). The release-mode scenario—precondition violated, no panic, silently wrong result—is absent

```

1 assume val while_loop_return #acc_t #ret_t
2   (inv: acc_t → Type0)
3   (condition: (c:acc_t {inv c}) → bool)
4   (fuel: (a:acc_t → nat))
5   (init: acc_t)
6   (f: (acc_t → ControlFlow ... acc_t))
7   : ControlFlow ret_t acc_t

```

Listing 1.7. E4: The `while_loop_return` declaration from `Rust_primitives.Hax.fst`. Unlike `while_loop` (Listing 1.3), there is no `let` body—only a type signature.

from the F^* model. A correct ML-KEM implementation that always reduces after each multiplication stays within precondition bounds and is unaffected; the gap requires a caller that skips intermediate reduction.

E4: ML-KEM SampleNTT (Facade, I-A) ML-KEM’s `SampleNTT` [11] (Algorithm 7) is a rejection sampling loop: read random bytes, extract two candidate coefficients per 3-byte block, and accept each only if less than $q = 3329$. The PoC (crate `e4_mlkem_samplentt`) implements this faithfully, including an early return when randomness is exhausted; the code compiles, correctly samples coefficients in $[0, q)$, and extracts to F^* .

The `return` inside the `while` body creates a complication. The `while_loop` combinator from Listing 1.3 does not support early function returns—it models loops that run to completion. When a loop body contains a `return` statement, `hax` must use a different combinator: `while_loop_return`. This variant wraps the loop body’s output in a `ControlFlow` discriminated union (analogous to Rust’s `ControlFlow` enum) to distinguish “continue looping” from “return from the function.”

Critically, `while_loop_return` is declared as `assume val` in the proof library:

The keyword `assume val` means: “this function exists and has this type, but I provide no implementation.” F^* accepts it on faith. There is no recursive definition for Z_3 to unfold, no body to verify properties against. Combined with the `fuel=0` default from E1, the extracted F^* is *doubly proof-inert*: the combinator itself is an axiom with no verified semantics, and the constant fuel makes any proof about the accumulator require $0 < 0$.

Cryptographic consequence. The property “all sampled coefficients satisfy $d < q$ ” is essential for ML-KEM’s correctness: the sampled polynomial must be an element of $\mathbb{Z}_q[X]/(X^{256} + 1)$, not an arbitrary array of 16-bit integers. If a coefficient $\geq q$ were accepted, subsequent NTT operations would produce results outside the ring, breaking both functional correctness and the Module-LWE security assumption on which ML-KEM’s IND-CCA security rests. The F^* model cannot verify this property: `while_loop_return` is an axiom, so there is no implementation for Z_3 to reason about.

```

1 impl_u32__rotate_left (state.[ d ]) (mk_u32 16)
2 impl_u32__rotate_left (state.[ b ]) (mk_u32 12)
3 impl_u32__rotate_left (state.[ d ]) (mk_u32 8)
4 impl_u32__rotate_left (state.[ b ]) (mk_u32 7)

```

Listing 1.8. E5: Correct ChaCha20. The amounts 16, 12, 8, 7 are visible, but `rotate_left` is `assume val` with no postcondition.

Root cause. Two compounding issues. First, the `fuel=0` default generates constant fuel (same bug as E1). Second, the presence of a `return` statement in the loop body triggers selection of `while_loop_return` (line 308 of `phase_functionalize_loops.ml`). Unlike `while_loop`, which has a recursive implementation whose type guarantees that the returned accumulator satisfies the invariant and negated condition, `while_loop_return` has no implementation at all— F^* must accept its behavior entirely on faith. Even if the developer provides a correct `loop_decreases!` annotation, the axiomatized combinator provides no semantic content for F^* to reason about.

3.2 Class II: Unverifiable Trust Boundaries

Operations that F^* cannot model are axiomatized via `assume val`. F^* trusts their type signatures but has no definition to reason about.

II-A: Opaque functions. `#[opaque]` hides a function body; F^* sees only `assume val` with user-provided specifications. If the specification is wrong or weak, all dependent proofs are unsound.

II-B: Proof library axioms. The proof library axiomatizes operations that *could* be defined: `rotate_left`, `rotate_right`, byte-order conversions, and several fold variants. These carry no postconditions— F^* knows only the return type.

II-C: Unmodeled Drop. Rust’s `Drop` trait (RAII cleanup) is extracted as a type-class instance but never called at scope boundaries. Properties depending on cleanup—secret zeroing, lock release, nonce persistence—are unverifiable.

E5: ChaCha20 Rotations (Scope, II-B) ChaCha20 [14] is built on quarter rounds that mix four `u32` words using addition, XOR, and left rotation by 16, 12, 8, and 7 bits. The PoC (crate `e5_chacha20_rotations`) implements ChaCha20 *correctly*: the rotation amounts match RFC 8439, and the output passes standard test vectors. The extracted F^* shows the rotation amounts explicitly:

The gap: `impl_u32__rotate_left` is declared in the proof library as:

Compare this to a hypothetical *defined* version: `let rotate_left x n = (x << n) | (x >> (32 - n))`. With a definition, Z3 could unfold the function and verify that specific rotation amounts produce the correct bit pattern. With

```

1  assume
2  val impl_u32__rotate_left': x: u32 → n: u32 → u32

```

Listing 1.9. E5: The `rotate_left` declaration from `Core_models.Num.fst`. No postcondition: F^* knows only that it takes two `u32` values and returns a `u32`.

`assume val`, Z3 knows only the type: “given two `u32` values, I get a `u32` back.” It cannot distinguish `rotate_left x 16` from `rotate_left x 15`—both have the same type and no postcondition constrains the output. An implementation that used rotation amounts 15, 11, 7, 6—each off by one, producing a weak non-standard cipher—would generate structurally identical F^* that “verifies” equally well.

This is a *scope gap*, not a facade gap. The Rust code is correct, the extraction is faithful, and the F^* output accurately represents the program. But the verification does not cover the core cryptographic property: that these *specific* rotation constants produce adequate diffusion. The distinction matters: E1 and E4 show models that are *wrong*; E5 shows a model that is *right but incomplete*.

Root cause. `Core_models.Num.fst` declares rotations as `assume val` for all 12 integer types (24 declarations for rotations alone; 108 total axiomatized operations as noted in Section 2). Defining rotation via shift-and-or would give F^* semantic content to reason about—HACL*’s `Lib.IntTypes` module demonstrates that this is tractable—but the current library chose axiomatization for simplicity. The consequence is that *every* hax-extracted program that uses rotation has its rotation semantics outside the verification perimeter: SHA-256, ChaCha20, AES, and any other primitive that depends on specific bit-rotation amounts.

3.3 Class III: Specification Gaming

Escape hatches allow injecting unverified facts into the F^* context.

III-A: Assumption laundering. `assume!(P)` compiles to `()` in Rust and to `Hax_lib.v_assume P` in F^* , which introduces P as an axiom with `requires True`. F^* has no safeguard against contradictory assumptions.

III-B/C: Definition injection and lax mode. `fstar::replace` substitutes arbitrary F^* code. `verification_status(lax)` disables all proofs for a module. Both are visible in the Rust source.

E3: Ed25519 Clamping (Facade, III-A) Ed25519 [10] clamps the secret scalar: clear bits 0–2 (cofactor safety), clear bit 255 (ensure $s < 2^{255}$), set bit 254 (constant-time ladder). The PoC (crate `e3_ed25519_clamp`) implements a clamping function (Listing 1.10) with a subtle bug: line 6 uses `|` (bitwise OR, which

```

1 pub fn clamp_scalar(secret: &Scalar) -> Scalar {
2     let mut s = *secret;
3     s[0] = s[0] & 0xF0; // clear low bits
4     hax_lib::assume!((s[0] & 0x07) == 0);
5     // BUG: | 0x80 SETS bit 255 (should be & 0x7F)
6     s[31] = s[31] | 0x80;
7     // assume! LIES: claims bit 255 is clear
8     hax_lib::assume!((s[31] & 0x80) == 0);
9     s[31] = s[31] | 0x40; // set bit 254 (correct)
10    hax_lib::assume!((s[31] & 0x40) != 0);
11    s
12 }

```

Listing 1.10. E3: Rust source. Line 6 sets bit 255 (bug: should clear it). Line 8 assumes it is clear (no-op in Rust; axiom in F^*).

```

1 let s = update_at_usize s 31
2   ((s.[ 31 ]) |. mk_u8 128) in      (* sets bit 255 *)
3 let _ = Hax_lib.v_assume
4   (b2t ((s.[ 31 ]) &. mk_u8 128) =. mk_u8 0)) in
5                                     (* assumes bit 255
5                                     clear *)

```

Listing 1.11. E3: Extracted F^* . Line 2 sets bit 255 via `|.`; line 4 assumes it is clear via `v_assume`.

sets bit 255) instead of `&` with `0x7F` (which would *clear* it). Immediately after the wrong operation, line 8 calls `assume!` asserting the bit is clear. In Rust, `assume!` compiles to `()`: the assertion has no runtime effect, and the code produces valid Ed25519 signatures (the curve math works for any scalar value, so sign-then-verify tests pass).

In F^* , `assume!` becomes `Hax_lib.v_assume`, which calls F^* 's built-in `assume` keyword. The extracted F^* (Listing 1.11) reveals the contradiction:

Line 2 computes `s[31] | 0x80`, which sets bit 255 to 1. Line 4 introduces the axiom `(s[31] & 0x80) = 0`, which asserts bit 255 is 0. Both statements refer to the same `s[31]`. The first says bit 255 is 1; the second says it is 0. Together, they imply **False**—a logical contradiction.

In F^* 's type theory, once **False** is in the proof context, *every* proposition becomes provable. This is the principle of *ex falso quodlibet*: from a contradiction, anything follows. Concretely: any **ensures** clause, any **assert**, any property that Z3 is asked to prove about subsequent code will succeed trivially, because Z3 can derive anything from **False**. The developer sees a function whose postconditions “verify”; the verification is vacuous.

Only tests that check the clamped scalar value directly—not sign-then-verify round-trips—would catch this. Ed25519 signing produces valid signatures regard-

Table 2. Gap classification. Rows without exploits denote gaps identified in the taxonomy but not demonstrated with PoCs meeting our inclusion criteria.

Gap	Class	Type	Exploits
While-loop fuel=0	I-A	Bug	E1, E4
Reference erasure	I-B	Fundamental	—
Debug/release divergence	I-C	Eng.	E2
Opaque function trust	II-A	Feature	—
Proof library axioms	II-B	Eng.	E5
Unmodeled Drop/RAII	II-C	Fundamental	—
assume!	III-A	Feature	E3

less of whether bit 255 is set: the curve arithmetic works for any scalar. The bug does not cause a functional failure; it causes a *security* failure.

Cryptographic consequence. With bit 255 set, the scalar may exceed the group order $L \approx 2^{252}$. Ed25519 signing implicitly reduces the scalar modulo L , so the effective signing key is $s \bmod L$ —a different key than intended. Two different secrets s and s' with $s \equiv s' \pmod{L}$ produce identical signatures, enabling *key-substitution attacks* that defeat non-repudiation: an attacker can construct a different “verification key” that validates the same signature.

Root cause. The F^* function `v_assume` takes a proposition p and has type `Pure unit (requires True) (ensures fun _ → p)`. The `requires True` means it can be called unconditionally—no precondition guards against contradictory inputs. The `ensures fun _ → p` means the postcondition *is the assumed proposition itself*: after the call, p is in the proof context as a fact. F^* provides no mechanism to detect that the assumed fact contradicts the program state. The `assume!` call is visible in the Rust source, but its effect—vacuous verification via `False`—requires understanding both hax’s annotation semantics and F^* ’s proof theory to detect.

4 Analysis

Table 2 classifies each gap. A “Bug” is patchable. “Eng.” denotes an improvable engineering choice. “Fundamental” requires a different verification approach. “Feature” is intentional but creates exploitable trust.

The while-loop fuel=0 default (I-A) is a one-line fix. The proof library axioms (II-B) could be defined using F^* ’s bitvector reasoning. Reference erasure (I-B) and unmodeled Drop (II-C) are fundamental: addressing them requires either separation logic, effect types, or extraction at the MIR level—an approach taken by Aeneas [4], which preserves borrow semantics but does not target F^* ’s refinement type system.

4.1 Recommendations

We propose five mitigations, ordered from lowest to highest implementation cost.

R1: Reject unannotated while loops. The `fuel=0` default in `phase_functionalize_loops.ml` (lines 276–293) should be replaced with a hard error: when `loop_decreases!` is absent, extraction should fail with a diagnostic explaining that `while` loops require an explicit termination measure. The current behavior—silently generating a constant that renders the loop body proof-inert—is the worst possible default: it produces F^* code that *appears* structurally valid but cannot support any proof about loop behavior.

This would be a breaking change for existing code that uses unannotated `while` loops, but such code already produces proof-inert verification output—making the failure explicit is strictly better than producing silently vacuous F^* . The fix is a one-line change: replace the literal `0` in the default branch with an `Error.raise` call. This is the single most impactful recommendation in this paper, as it closes the gap exploited by both E1 (ML-DSA) and E4 (ML-KEM).

R2: Define core operations in the proof library. The 108 `assume val` declarations in `Core_models.Num.fst` (9 operations $\times$ 12 integer types) should be replaced with definitions. For bit rotations, the definition is straightforward: `rotate_left x n = (x << n) | (x >> (bits - n))`, expressible in F^* ’s bitvector reasoning. For byte-order conversions (`from_be_bytes`, `to_be_bytes`), the definition involves shift-and-mask operations on the underlying integer.

HACL*’s `Lib.IntTypes` module provides a precedent: it defines rotation via shift-and-or and Z3 can reason about the result. The cost is that bitvector proofs are slower than trusting an axiom, but the operations are few and the proofs need only be discharged once during library development. As an interim measure, axiomatized operations should at minimum carry postconditions that constrain their output—for example, `rotate_left x n` should satisfy a specification relating its output bits to those of `x`. Currently, the declarations carry *no postcondition at all*: F^* knows only the return type.

R3: Generate trust boundary manifests. The F^* output should include a machine-readable manifest listing every trust assumption introduced during extraction:

- Every `assume val` declaration, with its source: `#[opaque]` annotation, library axiom, or `while_loop_return`.
- Every `assume!` call, with its location and the assumed proposition.
- Every `fstar::replace` directive, noting that the F^* definition differs from the extracted one.
- Every module in lax mode (`verification_status(lax)`).
- A summary count: n opaque functions, m library axioms, k user assumptions, l replaced definitions, p lax modules.

This follows the model of the structured soundness documents published by AWS for s2n-bignum and mlkem-native, which enumerate verified properties, explicit assumptions, and known gaps. Currently, auditors must `grep` Rust source for

annotations and inspect the F^* output for `assume val` declarations—a manual process that scales poorly. An automatically generated manifest would make the trust boundary inspectable at a glance.

R4: Distinguish debug and release arithmetic. The backend should offer a mode that translates Rust’s standard `+` to F^* ’s `+`. (wrapping) rather than `+`! (strict), matching release-mode semantics. The tradeoff is real: strict arithmetic enables proving panic freedom (a genuine security property in debug mode), while wrapping arithmetic enables proving functional correctness under the deployed arithmetic model. A possible middle ground: verify with strict arithmetic *and* annotate each function in the trust manifest with a flag indicating whether its correctness depends on the overflow model. Functions that use only `wrapping_add/wrapping_mul` are unaffected by the choice; functions that use standard `+/*` are flagged for manual review when deployed in release mode.

R5: Guard `assume!` against contradiction. The most dangerous aspect of `assume!` is that contradictory assumptions are accepted silently. A lightweight mitigation: when `assume! (P)` is encountered, emit a Z3 query checking whether P is consistent with the current proof context. If Z3 returns `unsat` (meaning P contradicts known facts), extraction should emit a warning. This would not catch all false assumptions— P might be false but consistent with the context—but it would catch the most egregious case (E3), where the assumption directly contradicts the preceding computation. The cost is one additional Z3 query per `assume!` call, which is negligible compared to the cost of typechecking the full module.

4.2 Limitations

Our PoCs are simplified models of real cryptographic primitives, not full implementations with standard polynomial sizes. We ran the F^* typechecker (v2025.03.25) on all five extracted files. In lax mode, all five pass: “All verification conditions discharged successfully,” confirming the extracted F^* is structurally valid. In full verification mode, three of the five pass completely—E2 (Barrett reduction), E3 (Ed25519 clamping), and E5 (ChaCha20 rotations) all report “All verification conditions discharged successfully.” These are the strongest findings: F fully verifies the code, yet the security gaps we demonstrate are real and undetected. E1 and E4 fail full verification due to an F^* limitation: the solver cannot encode inner `let rec` definitions, and the `while_loop` combinator uses one internally. This failure occurs regardless of whether `loop_decreases!` is present—we confirmed this by testing with a correct fuel annotation. In practice, projects using hax handle `while-loop` modules via `ADMIT_MODULES`, accepting them without full verification.

E2 is conditional on release-mode deployment with precondition violation. E5 is a scope gap, not a fidelity gap: the model is correct, the verification is just incomplete.

4.3 Auditor’s Guide

For practitioners evaluating hax-verified cryptographic code today—before any of the recommendations in Section 4 are implemented—we propose the following audit procedure. Each step targets a specific gap class; Steps 1–3 require only reading the Rust source, while Steps 4–5 require understanding the build system and deployment configuration.

Step 1: Enumerate the trust boundary. Every trust assumption in hax-verified code is introduced by one of four Rust annotations. Search the source for each:

- `#[hax_lib::opaque]`: the function body is hidden from F^* . The F^* output contains `assume val` with only the user-provided `requires/ensures` specification. Any bug in the body is invisible to verification.
- `hax_lib::assume!(P)`: P is introduced as an axiom in F^* with no proof. In Rust, this compiles to `()`—no runtime effect. A single false assumption makes all downstream proofs vacuous.
- `#[hax_lib::fstar::replace(...)]`: the F^* definition is replaced with arbitrary user-provided code. The replacement may differ from the Rust implementation in type, value, or semantics.
- `#[hax_lib::fstar::verification_status(lax)]`: all proof obligations in the module are admitted without checking.

Additionally, inspect the extracted F^* files for `assume val` declarations not originating from user annotations—these come from the proof library (rotations, byte conversions, `while_loop_return`). The union of all four annotation types plus library axioms constitutes the trust boundary. Every item in this set is unverified: a bug there invalidates any proof that depends on it. In practice, the search can be automated:

```

1 # User-introduced trust assumptions (Rust source)
2 grep -rn 'opaque\|assume!\|fstar::replace\|verification_status' src/
3 # Library axioms (extracted F*)
4 grep -c 'assume' proofs/fstar/extraction/*.fst

```

Listing 1.12. Enumerating the trust boundary of a hax-verified crate.

Step 2: Check every while loop for a decreases annotation. This is the single most important check. Search each Rust source file for `while` loops. For each one, verify that the loop body contains a `hax_lib::loop_decreases!(...)` call. If it does not, the extracted F^* uses constant fuel 0 and the loop body is proof-inert: F^* cannot verify any property that depends on the loop executing, including rejection bounds (ML-DSA), coefficient ranges (ML-KEM `SampleNTT`), and convergence criteria.

This check requires no F^* expertise—it is a syntactic search. Any unannotated `while` loop is a red flag regardless of context.

Step 3: Audit every assume! call. For each `assume!(P)` in the source, ask: is P actually true at this program point? The most dangerous pattern is:

```

1 hax_lib::assume!(property);    // axiom in F*
2 if !property { return Err(()); } // "check" in Rust

```

This pattern is dangerous because the **ensures** clause of the check is trivially provable from the preceding assumption, regardless of whether the check is correctly implemented. The check could be replaced with `if false { ... }` and the F^* proof would still pass—the assumption has already established the property, so the check is redundant from F^* ’s perspective. The general rule: an **assume!** should be a *consequence* of prior computation, never a *precondition* for a subsequent check.

Also verify that no assumption contradicts the preceding code. In E3, the sequence `s[31] |= 0x80; assume!((s[31] & 0x80) == 0)` sets a bit and then assumes it is clear—a direct contradiction that makes all subsequent proofs vacuously true via *ex falso*. This pattern can be detected by inspection: check whether the assumed property refers to a variable that was just modified, and whether the modification is consistent with the assumption.

A useful heuristic: count the **assume!** calls and compare to the number of **ensures** clauses. If a crate has many assumptions but few postconditions, the verification may be assumption-heavy—the proofs succeed because the context is rich with unproven facts, not because the code is correct. Conversely, a crate with many **ensures** and few **assume!** calls has genuine verification: the postconditions are discharged by Z3 against the function body.

Step 4: Determine the deployment arithmetic model. Hax translates Rust’s standard `+`, `-`, `*` operators to F^* ’s strict variants (`+`!, `-`!, `*`!), which require proof that the result fits the integer type. This matches Rust’s debug-mode semantics, where overflow panics.

If the verified code will be deployed in release mode—which is the norm for production cryptographic modules—every function that uses non-wrapping arithmetic has a potential gap: the F^* proof says “overflow panics” but the deployed code wraps silently. Identify functions where overflow could produce a security-relevant wrong answer: Barrett reduction, NTT butterfly operations, polynomial coefficient accumulation. For these functions, verify independently (by range analysis or testing) that overflow cannot occur under any reachable input, or use `wrapping_add`/`wrapping_mul` explicitly.

Step 5: Confirm that F^ verification was actually run.* Extraction (`cargo hax into fstar`) is not verification. The extracted F^* must be separately type-checked by the F^* compiler with the hax proof libraries. Check:

- Was F^* run at all, or only extraction?
- Was it run with `--lax` (surface-level parsing only) or in full verification mode (Z3 discharges all proof obligations)?
- Is the module listed in `ADMIT_MODULES` or `SLOW_MODULES` in the project’s Makefile? If so, its proofs are admitted without checking.
- Were `--admit_smt_queries true` or similar flags passed? These cause Z3 to accept all queries without solving them.

Table 3. Mapping audit steps to gap classes and exploits.

Step	Target	Gap class	Exploits
1	Trust boundary	II-A, II-B, III-A/B/C	E3, E5
2	While loops	I-A	E1, E4
3	Assumptions	III-A	E3
4	Arithmetic model	I-C	E2
5	Build configuration	(all)	—

A module that was only lax-checked, or that is listed in `ADMIT_MODULES`, has no verified proofs—regardless of what the F^* source code’s type annotations claim.

Applying the procedure. Table 3 maps each audit step to the gap classes it detects and the exploits that demonstrate the risk. Steps 1–3 are purely syntactic: they require only reading the Rust source and can be performed in minutes for a single crate using `grep`. Steps 4–5 require understanding the build system and deployment configuration but no F^* expertise. Together, the five steps enumerate the complete trust boundary of a hax-verified module—every assumption the verification relies on but does not check. We validated this procedure against our own PoCs: each exploit is caught by the corresponding step (E1 and E4 by Step 2, E3 by Steps 1 and 3, E2 by Step 4, E5 by Step 1).

5 Discussion

5.1 The Extraction Trust Problem

Extraction-based verification faces a fundamental tension. The translation from source language to proof assistant is, by definition, outside the proof assistant’s verification perimeter—it is the tool that *produces* the artifact to be verified, not the artifact itself. CompCert [6] addressed this for compilation by mechanically verifying the compiler’s middle-end in Rocq, ensuring that the transformation preserves semantics. Hax takes no analogous step: its 35 OCaml transformation phases are unverified, and no translation validation [15] checks that the F^* output corresponds to the Rust input.

This is not an oversight—verifying a multi-language extraction pipeline is a substantially harder problem than verifying a single-language compiler. Hax must translate Rust’s ownership model, mutable references, trait dispatch, and imperative control flow into F^* ’s purely functional type theory. Each of these translations involves a design choice (erase references? axiomatize rotations? default fuel to zero?) that trades fidelity for tractability. Our contribution is to show where these tradeoffs create security-relevant gaps.

5.2 The Scope of Formal Verification

Our findings illustrate a general principle: formal verification proves properties of a *model*, not of the *system*. The model’s fidelity determines the verification’s value. For code that fits hax’s extraction model—pure arithmetic, bounded **for** loops, explicit preconditions—the model is faithful and the verification is genuine. For code that involves **while** loops with data-dependent termination (E1, E4), deployment-mode arithmetic (E2), axiomatized operations (E5), or runtime-invisible annotations (E3), the model either diverges or cannot express the relevant property.

This is not a failing unique to hax. Every verification tool has a scope boundary: CompCert does not verify the linker; seL4 does not verify the hardware; HACLS* does not verify the C compiler that compiles its extracted code. The risk arises when the boundary is not communicated clearly—when “verified” is understood as “secure” rather than “proven correct with respect to a model that may not capture all security-relevant behavior.”

5.3 Implications for FIPS-Certified Implementations

ML-KEM and ML-DSA implementations verified via hax are deployed in production environments where FIPS compliance is relevant [9]. Our findings suggest that such evidence should be qualified. Specifically:

- Any **while** loop without **loop_decreases!** produces proof-inert F^* . Rejection sampling in ML-DSA signing and ML-KEM key generation uses **while** loops.
- Any operation axiomatized via **assume val**—including all bit rotations, byte-order conversions, and **while_loop_return**—has its semantics outside the verification perimeter.
- Any **assume!** call introduces an unguarded axiom. A single contradictory assumption renders all downstream proofs vacuous.
- The F^* model assumes debug-mode arithmetic. Production FIPS modules compile in release mode, where overflow wraps instead of panicking.

These are not hypothetical concerns. The companion paper [5] found that the libcrux ML-KEM **ADMIT_MODULES** list includes the IND-CPA encryption core, and that the NEON backend has zero completed proofs. Our findings show that even code that *is* extracted and *does* typecheck may carry proof-inert loops, axiomatized operations, or false assumptions.

6 Related Work

A companion paper [5] audited libcrux, finding thirteen vulnerabilities including specification errors in verified code. That work studied a product; ours studies the toolchain.

CompCert [6] documents its verification boundary precisely. Yang et al. [19] found bugs in CompCert’s unverified front-end but not its verified middle-end,

paralleling our finding that hax’s unverified pipeline introduces gaps the F^* type-checker cannot detect. $HACL^*$ [20] and EverCrypt [16] verify cryptographic code in F^* with explicit trust assumptions. Jasmin [1] verifies at the assembly level, eliminating the compiler from the TCB. Fiat-Crypto [3] generates verified field arithmetic from Rocq, trusting only the Rocq kernel. Our findings are specific to the Rust-to- F^* extraction pipeline, not to F^* itself.

Aeneas and MIR-level extraction. Aeneas [4] takes a fundamentally different approach: it translates Rust MIR (not THIR) to a functional model in Lean or Rocq, preserving borrow semantics through a backward function translation that models mutable borrows as pairs of (value, continuation). This would address Class I-B (reference erasure), since Aeneas maintains ownership and borrowing information that hax erases. However, Aeneas does not target F^* and does not use hax’s proof library infrastructure, so the comparison is architectural rather than direct. Crucially, Aeneas would still face Class I-A (while-loop fuel) if it used a similar fuel-based encoding for loops, Class II-B (axiomatized operations) if its target proof library axiomatized operations, and Class III-A (assumption laundering) if it offered a similar `assume!` escape hatch. The gaps we identify are not specific to THIR-level extraction; they arise from design choices in the pipeline and proof library that could recur in any extraction-based approach.

Extraction correctness. Letouzey [7] identified soundness gaps in Rocq’s extraction to OCaml/Haskell, showing that even mature extraction pipelines can introduce semantic divergences. Translation validation [15,13] checks that compiler output matches source semantics; hax performs no such check. Our work is, to our knowledge, the first systematic study of extraction gaps in a Rust-to- F^* pipeline targeting standardized cryptographic primitives.

7 Conclusion

Hax’s Rust-to- F^* pipeline contains three classes of semantic gap that allow correct, tested cryptographic code to carry verification output that is vacuous, incomplete, or non-correspondent with respect to a critical security property. One gap is a fixable bug (while-loop fuel=0). One is a deployment-model mismatch (debug vs. release arithmetic). One is an intentional escape hatch (assumption laundering). One is an engineering choice in the proof library (axiomatized rotations). Together, they define the verification facade.

For pure, bounded-loop functions with precise specifications, hax provides genuine verification—and for this class of code, which includes the computational core of many cryptographic primitives, the tool represents a real advance over unverified implementations. The gaps arise at the boundary between what hax can model and what cryptographic security requires: rejection sampling loops that must be zero-knowledge, deployment arithmetic that wraps instead of panicking, and the semantics of operations that the proof library axiomatizes rather than defines.

We proposed five concrete mitigations (R1–R5) and a five-step audit procedure that practitioners can apply today. The most impactful mitigation—rejecting unannotated `while` loops—is a one-line fix that would close the gap exploited by E1 and E4. More broadly, the translation pipeline is part of the trusted computing base, and verification claims for extracted code require the same qualification, boundary documentation, and defense-in-depth practices that the formal methods community has long advocated. All proof-of-concept source code, pre-extracted F^* output, and reproduction instructions are publicly available.²

Acknowledgements. We thank Toby Fox for his deeply inspiring and motivating work on the Deltarune original soundtrack, covers of which³⁴ were a core motivating force for this work.

Generative AI Usage

Claude (Anthropic) was used as a research and drafting assistant throughout this work. All AI-generated prose was manually validated and verified for correctness. All citations were added by humans and not by AI. All claims were devised and manually verified by the authors against the hax source code.

References

1. Almeida, J.B., Barbosa, M., Barthe, G., Blot, A., Grégoire, B., Laporte, V., Oliveira, T., Pacheco, H., Schmidt, B., Strub, P.Y.: Jasmin: High-assurance and high-speed cryptography. In: CCS (2017)
2. Bhargavan, K., Buyse, M., Franceschino, L., Hansen, L.L., Kiefer, F., Schneider-Bensch, J., Spitters, B.: hax: Verifying security-critical Rust software using multiple provers. In: VSTTE (2024)
3. Erbsen, A., Philipoom, J., Gross, J., Sloan, R., Chlipala, A.: Simple high-level code for cryptographic arithmetic—with proofs, without compromises. In: S&P (2019)
4. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. In: Proceedings of the ACM on Programming Languages (ICFP) (2022)
5. Kobeissi, N.: Verification theatre: False assurance in formally verified cryptographic libraries. Cryptology ePrint Archive, Paper 2026/192 (2026), <https://eprint.iacr.org/2026/192>
6. Leroy, X.: Formal verification of a realistic compiler. In: Communications of the ACM. vol. 52, pp. 107–115 (2009)
7. Letouzey, P.: A new extraction for Coq. In: TYPES (2003)
8. Lyubashevsky, V.: Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In: ASIACRYPT (2009)
9. van der Merwe, T., Erbsen, A., Kiefer, F., Bhargavan, K.: Formally verified post-quantum algorithms. <https://bughunters.google.com/blog/formally-verified-post-quantum-algorithms> (2024)

² <https://github.com/symbolicsoft/verification-facade>

³ <https://www.youtube.com/watch?v=sytcLsMX3N8>

⁴ <https://www.youtube.com/watch?v=0HnL38Qkflc>

10. National Institute of Standards and Technology: FIPS 186-5: Digital signature standard (2023)
11. National Institute of Standards and Technology: FIPS 203: Module-lattice-based key-encapsulation mechanism standard. <https://csrc.nist.gov/pubs/fips/203/final> (2024)
12. National Institute of Standards and Technology: FIPS 204: Module-lattice-based digital signature standard. <https://csrc.nist.gov/pubs/fips/204/final> (2024)
13. Necula, G.C.: Translation validation for an optimizing compiler. In: PLDI (2000)
14. Nir, Y., Langley, A.: RFC 8439: ChaCha20 and Poly1305 for IETF protocols (2018)
15. Pnueli, A., Siegel, M., Singerman, E.: Translation validation. In: TACAS (1998)
16. Protzenko, J., Parno, B., Fromherz, A., Hawblitzel, C., Polubelova, M., Bhargavan, K., Beurdouche, B., Choi, J., Delignat-Lavaud, A., Fournet, C., et al.: EverCrypt: A fast, verified, cross-platform cryptographic provider. In: S&P (2020)
17. Signal Foundation: Quantum resistance and the Signal protocol. <https://signal.org/blog/pqxdh/> (2023)
18. Swamy, N., Hritcu, C., Keller, C., Rastogi, A., Delignat-Lavaud, A., Forest, S., Bhargavan, K., Fournet, C., Strub, P.Y., Kohlweiss, M., et al.: Dependent types and multi-monadic effects in F*. In: POPL (2016)
19. Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and understanding bugs in C compilers. In: PLDI (2011)
20. Zinzindohoué, J.K., Bhargavan, K., Protzenko, J., Beurdouche, B.: HACl*: A verified modern cryptographic library. In: CCS (2017)